

Sincronización entre Procesos

Sistemas Operativos 1151018



Realiza lo siguiente

- A los programas de escritor y lector originales de memoria compartida, añade un nuevo programa (incrementa.c).
- Inicializa el arreglo con todos los elementos en ceros con tu escritor.
- Luego ejecuta dos veces el programa (incrementa) al mismo tiempo: `./incrementa & ./incrementa & wait`
- Después ejecuta el lector.
- ¿Qué pasó con los valores del arreglo? ¿Tienen coherencia? ¿Por qué?



1

```
#include<stdio.h>
#include<stdlib.h>
#include<sys/ipc.h>
#include<sys/shm.h>

#define TAM 5
#define REP 100000

int main(){
    key_t clave;
    int id_memoria;
    int *arreglo;

    clave = ftok("/tmp", 'A');
    if(clave == -1){
        perror("Error al crear la llave");
        exit(1);
    }

    id_memoria = shmget(clave, TAM*sizeof(int), 0666);
    if(id_memoria == -1){
        perror("Error al conectarse a la memoria compartida");
        exit(1);
    }

    arreglo = (int *)shmat(id_memoria, NULL, 0);
    if(arreglo == (int *)(-1)){
        perror("Error al adjuntar el segmento");
        exit(1);
    }
}
```

2

```
for(int r=0; r<REP; r++){
    for(int i=0; i<TAM; i++){
        arreglo[i] = arreglo[i] + 1;
    }
}

shmdt(arreglo);

return 0;
}
```

1

```
Terminal - alumnado@debianSO: ~/Documentos/programas
Archivo Editar Ver Terminal Pestañas Ayuda
alumnado@debianSO:~/Documentos/programas$ ./escritor_compartida
Ingresa los elementos del arreglo
0 0 0 0 0

Arreglo almacenado en memoria compartida
alumnado@debianSO:~/Documentos/programas$ ipcs -m

---- Segmentos memoria compartida ----
key          shmid      propietario perms      bytes      nattch     estado
0x00000000  10           alumnado    600       524288     2          dest
0x00000000  15           alumnado    600       524288     2          dest
0x00000000  18           alumnado    600       524288     2          dest
0x00000000  23           alumnado    600       4194304    2          dest
0x00000000  27           alumnado    600       1048576    2          dest
0x00000000  39           alumnado    600       524288     2          dest
0x00000000  32813        alumnado    600       524288     2          dest
0x00000000  46           alumnado    600       524288     2          dest
0x00000000  49           alumnado    600       524288     2          dest
0x4101fa04  32818        alumnado    666       20         0          dest
0x00000000  52           alumnado    600       524288     2          dest
0x00000000  55           alumnado    600       4399104    2          dest
0x00000000  56           alumnado    600       3960832    2          dest
```

2

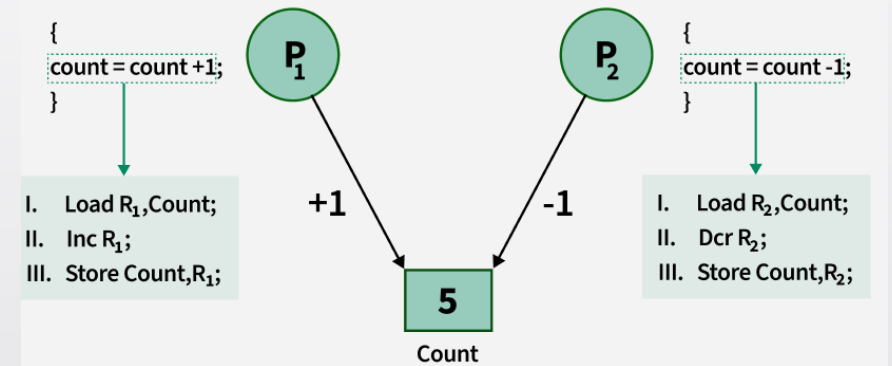
```
Terminal - alumnado@debianSO: ~/Documentos/programas
Archivo Editar Ver Terminal Pestañas Ayuda
alumnado@debianSO:~/Documentos/programas$ ./incrementa & ./incrementa & wait
[1] 3939
[2] 3940
[1]- Hecho      ./incrementa
```

3

```
Terminal - alumnado@debianSO: ~/Documentos/programas
Archivo Editar Ver Terminal Pestañas Ayuda
alumnado@debianSO:~/Documentos/programas$ ./lector_compartida
Arreglo leído desde la memoria compartida:
169359 169471 169576 169471 169439
alumnado@debianSO:~/Documentos/programas$
```


La sincronización de procesos

- Es un mecanismo de los SO que coordina la ejecución de múltiples tareas concurrentes.
- Su objetivo principal es asegurar la **coherencia de datos**, evitar **condiciones de carrera** y **prevenir bloqueos** cuando distintos procesos compiten por un recurso compartido.



Problemas comunes

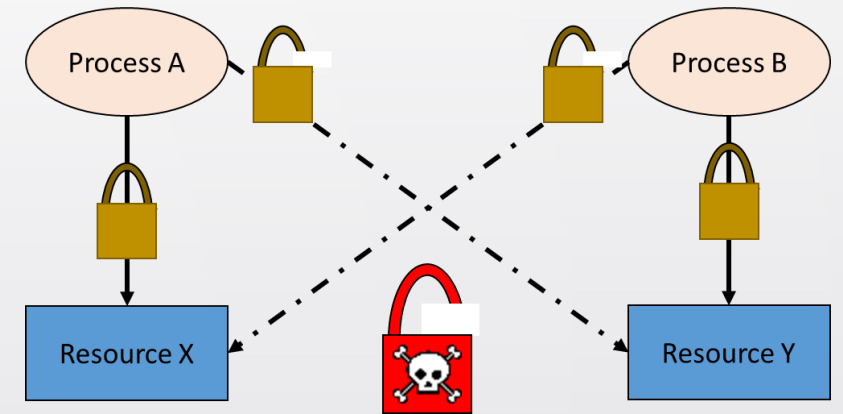
- **Condición de carrera:** Ocurre cuando varios procesos intentan acceder y modificar un dato al mismo tiempo, y el resultado final depende del proceso que finalice último.
- **Sección crítica:** Es el segmento de código donde un proceso accede a un recurso compartido. No se debe permitir que dos procesos estén en su sección crítica al mismo tiempo



Los Peligros de una Mala Sincronización

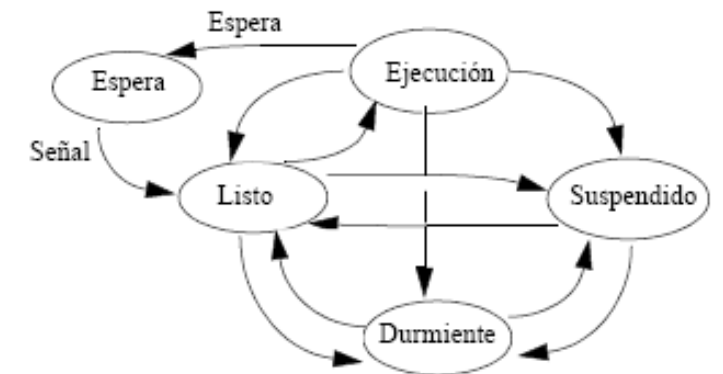
Si la sincronización no se diseña con cuidado, pueden ocurrir problemas graves:

- **Interbloqueo (Deadlock):** Ocurre cuando dos o más procesos se bloquean mutuamente esperando un recurso que tiene el otro. *Ejemplo:* El Proceso A tiene el Recurso 1 y quiere el 2; el Proceso B tiene el Recurso 2 y quiere el 1. Ninguno avanza jamás.
- **Inanición (Starvation):** Un proceso se queda esperando indefinidamente para entrar a su sección crítica porque el sistema operativo siempre le da prioridad a otros procesos.



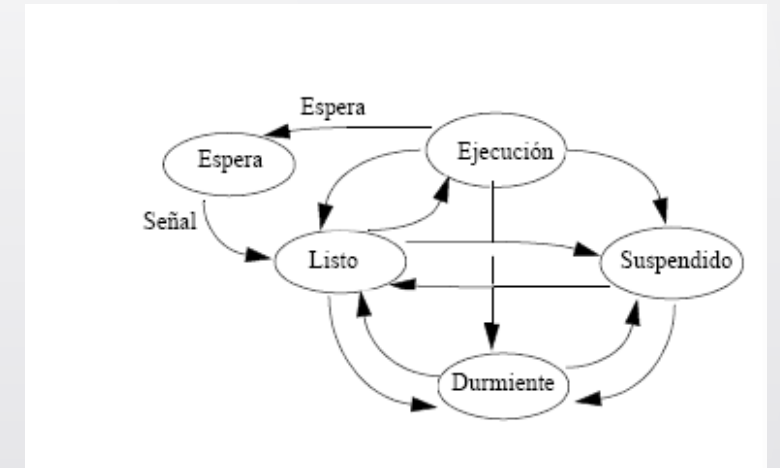
Mecanismos de sincronización

- **Semáforos:** Variables especiales (contador) que administran el acceso a uno o varios recursos, permitiendo o bloqueando el paso de los procesos.
- **Mutex (Mutual Exclusion):** Objeto de bloqueo que permite a un proceso tomar el control de un recurso, realizar su tarea y liberarlo para los demás



Mecanismos de sincronización

- **Exclusión Mutua:** Garantiza que solo un proceso a la vez acceda a un recurso o sección crítica.
- **Monitores:** Construcciones de programación de alto nivel que aseguran que solo un proceso pueda ejecutar código dentro del monitor a la vez, encapsulando datos y métodos



Semáforos

- Son uno de los mecanismos de sincronización más importantes en sistemas operativos. Fueron propuestos por **Edsger Dijkstra** en 1965.
- Su propósito principal es **sincronizar o coordinar** el acceso de **varios procesos** a recursos compartidos para evitar condiciones de carrera.



¿Qué es un Semáforo?

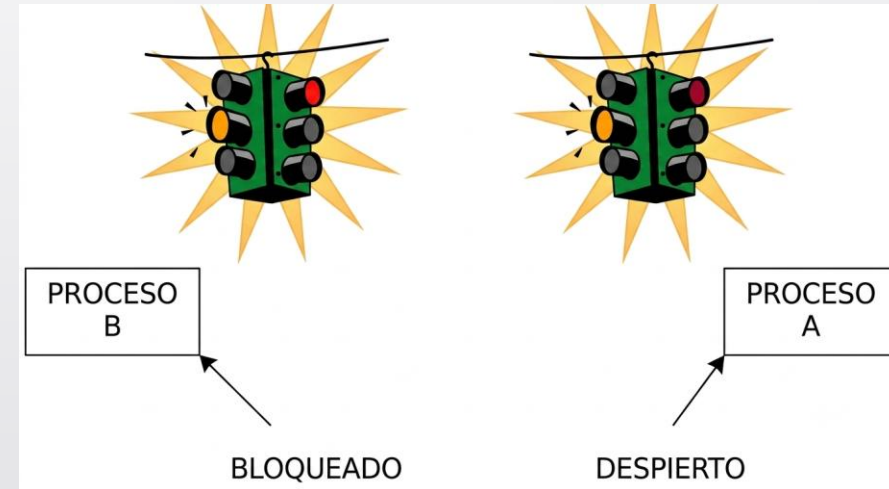
Es una variable entera no negativa que se maneja exclusivamente mediante dos operaciones:

- **wait()** o **P()** (Proberen = probar, intentar, disminuir)

verifica si un recurso está disponible; si no, bloquea la acción hasta que se libere.

- **signal()** o **V()** (Verhogen = incrementar)

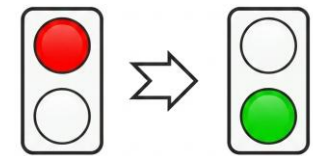
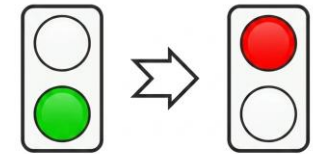
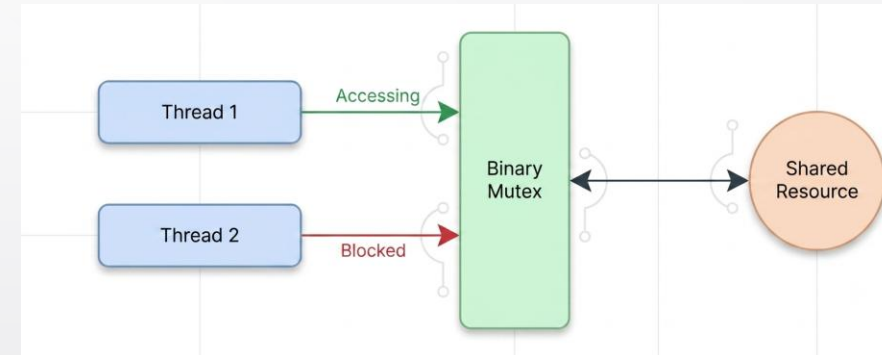
Es una operación que libera el recurso y notifica/desbloquea a otros procesos en espera.



Tipos de Semáforos (Binarios)

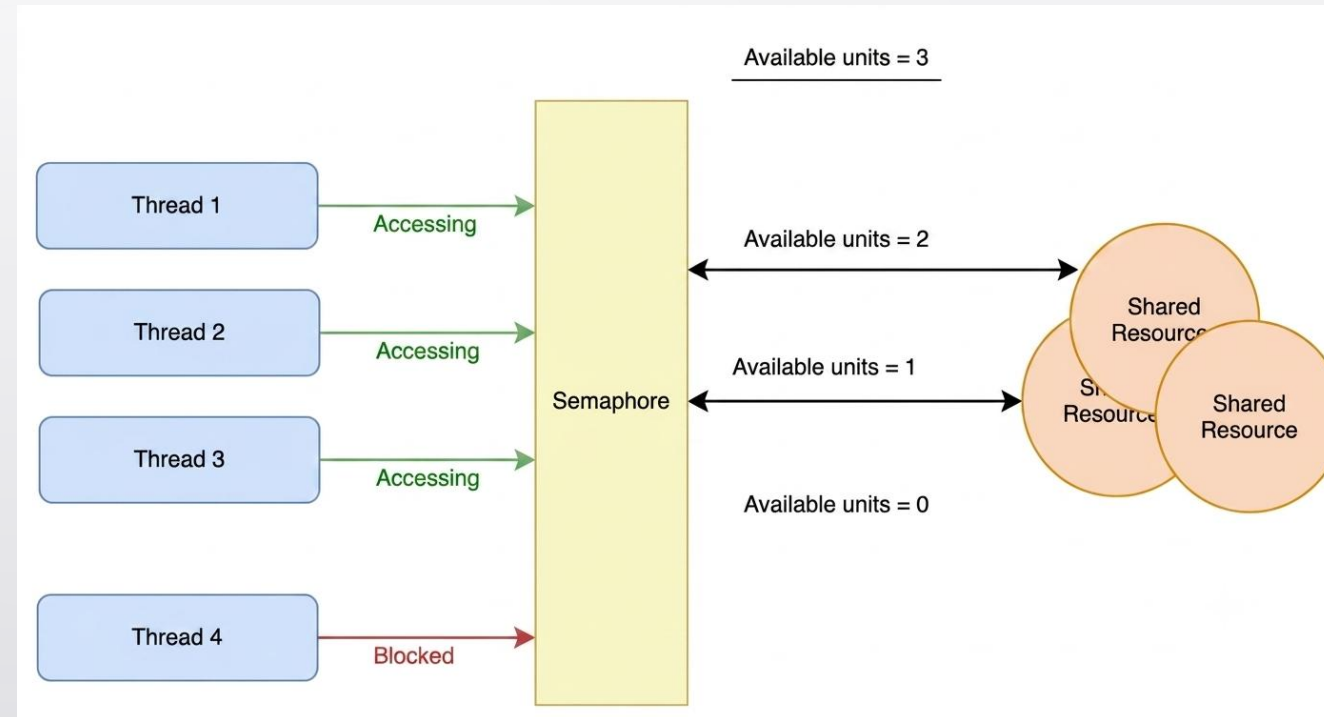
Son semáforos cuyo valor se restringe únicamente a **0** o **1**

- ✓ Controlan el acceso a un recurso compartido por un **solo proceso a la vez** (exclusión mutua).
- ✓ Si el valor es **1**, el recurso está **libre** y un proceso puede acceder a él cambiando el valor a 0.
- ✓ Si el valor es **0**, el recurso está **ocupado** y el proceso debe **esperar** a que se libere.



Tipos de Semáforos (conteo o generales)

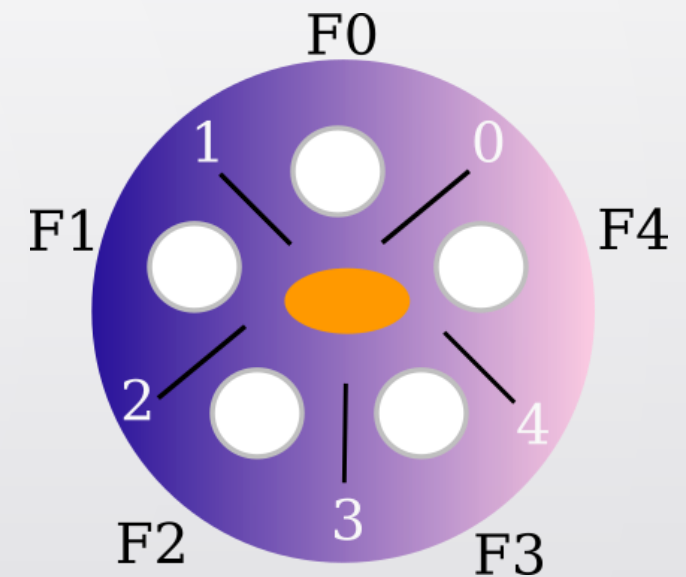
- ✓ Son semáforos que pueden tomar un valor entero **mayor o igual a 0**.
- ✓ Administran recursos que tienen múltiples unidades disponibles o **controlan el número de procesos** que pueden acceder a un recurso simultáneamente



Tipos de Semáforos (conteo o generales)

Cómo operan: El contador representa el número de recursos disponibles.

- Cada vez que un proceso usa el recurso, el semáforo se decrementa (operación **P** o *Wait*).
- Cuando un proceso libera el recurso, el semáforo se incrementa (operación **V** o *Signal*)





Semáforos

Elementos necesarios para trabajar con Semáforos

Elemento	Clasificación
ftok()	Función de biblioteca
semget()	Llamada al sistema IPC
semop()	Llamada al sistema IPC
semctl()	Llamada al sistema IPC

Semáforos semget()

Crear un conjunto de semáforos o conectarse a uno ya existente.

```
int semget(key_t key, int nsems, int semflg);
```

Parámetros:

1. **key**: Clave generada por ftok:
2. **nsems**: Número de semáforos que tendrá el conjunto.
3. **shmflg** = Permisos y opciones **IPC_CREAT | 0666**.

Valor de retorno

Devuelve el identificador del conjunto de semáforos.
-1 -> error

```
id sem=semget(clave, 1, IPC_CREAT|0666);
```


Semáforos semget()

Crear un conjunto de semáforos o conectarse a uno ya existente.

```
int semget(key_t key, int nsems, int semflg);
```

Parámetros:

1. **key**: Clave generada por ftok:
2. **nsems**: Número de semáforos que tendrá el conjunto.
3. **shmflg** = Permisos y opciones **IPC_CREAT | 0666**.

Valor de retorno

Devuelve el identificador del conjunto de semáforos.
-1 -> error

```
id sem=semget(clave, 1, IPC_CREAT|0666);
```

Semáforos semop()

- Realizar operaciones sobre uno o varios semáforos.
- Es la llamada que implementa las operaciones **wait (P)** y **signal (V)**

```
int semop(int semid, struct sembuf *sops, size_t nsops);
```

Parámetros:

1. **semid**: Identificador del conjunto de semáforos.
2. **sops**: Dirección de una estructura sembuf que describe la operación.
3. **nsops**: Número de operaciones a realizar.

Valor de retorno

Devuelve 0 si la operación fue exitosa.
-1 -> error

```
semop(id_sem, &operacion, 1);
```

Semáforos semop() estructura sembuf

El sistema operativo ya tiene definida esta estructura (en <sys/sem.h>) con tres campos clave:

```
struct sembuf {  
    unsigned short sem_num; // El índice del semáforo en el grupo (ej. 0 para el primero)  
    short          sem_op;  // La operación: un número negativo, positivo o cero  
    short          sem_flg; // Banderas opcionales (como SEM_UNDO)  
};
```

```
struct sembuf operacion;  
operacion.sem_num = 0; // Operar sobre el primer semáforo  
operacion.sem_op = -1; // Restar 1 (si está en 0, el programa se detiene a esperar)  
operacion.sem_flg = 0; // Sin banderas especiales  
  
semop(id_sem, &operacion, 1); // Ejecuta la operación
```

Semáforos semop() estructura sembuf

semop	Acción sobre el semáforo	¿El programa se puede bloquear?	Propósito común
Negativo (-1)	Resta al semáforo	Sí , si el semáforo está en 0.	Bloquear / Entrar a sección crítica.
Positivo (1)	Suma al semáforo	No , nunca se bloquea.	Liberar / Salir de sección crítica.
Cero (0)	No altera el valor	Sí , si el semáforo es mayor que 0.	Sincronizar (esperar a que todos terminen).

Semáforos semctl()

Inicializar, consultar, modificar o eliminar un conjunto de semáforos.

```
int semctl(int semid,int semnum,int cmd);
```

Parámetros:

1. **semid**: Identificador del conjunto de semáforos.
2. **semnum**: Número del semáforo dentro del conjunto.
3. **cmd**: Operación a realizar (**SETVAL**, **GETVAL**, **IPC_RMID**)

Valor de retorno

0.
-1 -> error

Union requerida



```
int r=semctl(id_sem, 0, SETVAL, arg);
```

Semáforos semctl()

Unión utilizada por semctl() para recibir información adicional dependiendo del comando que se ejecute.

```
union semun{  
    int val;  
    struct semid_ds *buf;  
    unsigned short *array;  
};
```

```
arg.val=1;
```

Val: Sirve para asignar un valor inicial al semáforo.

semid_ds: consultar permisos, propietario, fechas, estadísticas.

Array: Sirve cuando el conjunto tiene varios semáforos, inicializa todos de una sola vez.

```
int r=semctl(id_sem, 0, SETVAL, arg);
```



Ejemplo

Modifica tus programas **escritor** e **incrementa** con la adaptación de semáforos

2

1

```

#include<stdio.h>
#include<stdlib.h>
#include<sys/ipc.h>
#include<sys/shm.h>
#include<sys/sem.h>
#define TAM 5

// cambio, union para los semaforos semctl
union semun{
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};

int main(){

    key_t clave;
    int id_memoria, id_sem; //cambio
    int *arreglo;
    union semun arg; //cambio

    clave=ftok("/tmp", 'A');
    if (clave == -1) {
        perror("Error al crear la llave");
        exit(1);
    }

    id_memoria=shmget(clave, TAM*sizeof(int), IPC_CREAT|0666);
    if (id_memoria == -1) {
        perror("Error al crear la memoria compartida");
        exit(1);
    }
}

```

```

//cambio, crear 1 semaforo
id_sem=semget(clave, 1, IPC_CREAT|0666);
if(id_sem==-1){
    perror("Error al crear el semaforo");
    exit(1);
}

// inicializar el semaforo para que este disponible el recurso
arg.val=1; //valor de la union
int r=semctl(id_sem, 0, SETVAL, arg);
if(r==-1){
    perror("Error al inicializar el semaforo");
    exit(1);
}

//De aqui en adelante es lo mismo :D
arreglo=(int *)shmat(id_memoria, NULL, 0);
if (arreglo == (int *)(-1)) {
    perror("Error al Adjuntar el segmento al espacio de direcciones");
    exit(1);
}

printf("Ingresa los elementos del arreglo\n");
for(int i=0; i<TAM; i++){
    scanf("%d", &arreglo[i]);
}

printf("\nArreglo almacenado en memoria compartida\n");

//desadjuntar el segmento de memoria
shmdt(arreglo);
return 0;
}

```

1

```
#include<stdio.h>
#include<stdlib.h>
#include<sys/ipc.h>
#include<sys/shm.h>
#include<sys/sem.h>
```

```
#define TAM 5
#define REP 100000
```

```
int main(){
    key_t clave;
    int id_memoria, id_sem; //cambio
    int *arreglo;
    struct sembuf operacion; //cambio, explicar!!!

    clave = ftok("/tmp", 'A');
    if(clave == -1){
        perror("Error al crear la llave");
        exit(1);
    }

    id_memoria = shmget(clave, TAM*sizeof(int), 0666);
    if(id_memoria == -1){
        perror("Error al conectarse a la memoria compartida");
        exit(1);
    }

    //cambio, conectarse al semaforo creado
    id_sem = semget(clave, 1, 0666);
    if(id_sem == -1){
        perror("Error al conectarse al semaforo");
        exit(1);
    }
}
```

2

```
arreglo = (int *)shmat(id_memoria, NULL, 0);
if(arreglo == (int *)(-1)){
    perror("Error al adjuntar el segmento");
    exit(1);
}
```

```
//Cambios importantes
```

```
for(int r=0; r<REP; r++){

    /*Wait(p) bloqueo
    antes de entrar a la sección crítica
    */
    operacion.sem_num=0; //utilizar el semaforo con indice 0
    operacion.sem_op=-1; //importante, restar valor al semaforo (esperar)
    operacion.sem_flg=0; //bloqueo estandar
    semop(id_sem, &operacion, 1); //ejecuta las instrucciones

    //Entrar a la sección crítica (proteger con exclusion mutua)
    for(int i=0; i<TAM; i++){
        arreglo[i] = arreglo[i] + 1;
    }

    /*SIGNAL(V) Liberar recurso
    Salir de la sección crítica
    */
    operacion.sem_op=1; //incrementar el valor para liberar
    semop(id_sem, &operacion, 1);
}

shmdt(arreglo);

return 0;
}
```

incrementa_semaforos.c

Realiza lo siguiente

- Implementa el semáforo Binario
- Inicializa el arreglo con todos los elementos en ceros con tu escritor_semaforos.
- Ejecuta desde la terminal `ipcs -s`
- Luego ejecuta dos veces el programa (incrementa) al mismo tiempo:

`./incrementa_semaforos & ./incrementa_semaforos & wait`

- Después ejecuta el lector.
- ¿Qué pasó ahora con los valores del arreglo?



1

Salidas

27

```

Terminal - alumnado@debianSO: ~/Documentos/programas
Archivo  Editar  Ver  Terminal  Pestañas  Ayuda

alumnado@debianSO:~/Documentos/programas$ gcc escritor_semaforos.c -o escritor_semaforos

alumnado@debianSO:~/Documentos/programas$ ./escritor_semaforos
Ingresa los elementos del arreglo
0 0 0 0 0

Arreglo almacenado en memoria compartida
alumnado@debianSO:~/Documentos/programas$ ipcs -m

---- Segmentos memoria compartida ----
key          shmid      propietario perms    bytes    nattch   estado
0x00000000  8             alumnado   600      524288   2        dest
0x00000000  15            alumnado   600      524288   2        dest
0x00000000  18            alumnado   600      524288   2        dest
0x00000000  23            alumnado   600      4194304  2        dest
0x00000000  28            alumnado   600      1048576  2        dest
0x00000000  32800         alumnado   600      4399104  2        dest
0x00000000  32801         alumnado   600      3960832  2        dest
0x00000000  32803         alumnado   600      524288   2        dest
0x00000000  36            alumnado   600      524288   2        dest
0x4101fa04  32805         alumnado   666      20        0

```

```

Terminal - alumnado@debianSO: ~/Documentos/programas
Archivo  Editar  Ver  Terminal  Pestañas  Ayuda

alumnado@debianSO:~/Documentos/programas$ ipcs -s

----- Matrices semáforo -----
key          semid      propietario perms    nsems
0x4101fa04  0          alumnado   666      1

alumnado@debianSO:~/Documentos/programas$

```

3

```

Terminal - alumnado@debianSO: ~/Documentos/programas
Archivo  Editar  Ver  Terminal  Pestañas  Ayuda

alumnado@debianSO:~/Documentos/programas$ ./incrementa_semaforos & ./incrementa_
semaforos & wait
[1] 2947
[2] 2948
[1]- Hecho                ./incrementa_semaforos
[2]+ Hecho                ./incrementa_semaforos
alumnado@debianSO:~/Documentos/programas$ ./lector_compartida
Arreglo leído desde la memora compartida:
200000 200000 200000 200000 200000

```

2



Ejemplo

Crea un semáforo de conteo para simular núcleos de un procesador

Semáforo Binario vs Conteo

Característica	Semáforo Binario	Semáforo de Conteo
Valor Inicial (arg.val)	1	N (Cualquier entero positivo, ej. 2, 3, 5...)
Propósito Principal	Exclusión Mutua: en el acceso a variables o estructuras de datos no reentrantes.	Control de Capacidad: Limitar el acceso a un grupo con recursos finitos.
Procesos en Región Crítica	Máximo 1 proceso a la vez adentro.	Hasta "N" procesos al mismo tiempo adentro.
Analogía del mundo real	Un candado en la puerta de un baño público (ocupado/libre).	Los lugares disponibles en un estacionamiento público.
¿Cuándo se congela el WAIT?	Cuando el valor actual es 0 (ya hay alguien adentro).	Cuando el valor llega a 0 (todos los cupos se han agotado).


```
#include<stdio.h>
#include<stdlib.h>
#include<sys/ipc.h>
#include<sys/sem.h>
```

```
union semun{
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};

int main(){
    key_t clave;
    int id_sem;
    union semun arg;

    clave=ftok("/tmp", 'C');

    if(clave==-1){
        perror("Error al crear la llave");
        exit(1);
    }
    //crear el semaforo contador
    id_sem=semget(clave,1,IPC_CREAT|0666);

    if(id_sem == -1){
        perror("Error al crear el semaforo");
        exit(1);
    }
}
```

1

2

```
//Incializar el semaforo en 3 (simular 3 nucleos dsiponibles)
arg.val=3; //punto clave 3 recursos
int r=semctl(id_sem, 0, SETVAL, arg);

if(r==-1){
    perror("Error al incializar el semaforo");
    exit(1);
}

printf("Semaforo contador inicializado correctamente\n");
printf("ID del semaforo: %d\n", id_sem);
printf("Nucleos disponibles: %d\n",arg.val);

return 0;
}
```


2

1

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/sem.h>

struct sembuf operacion;

int main() {
    key_t clave;
    int id_sem, disponibles;

    clave = ftok("/tmp", 'C');
    if (clave == -1) {
        perror("Error al crear la llave");
        exit(1);
    }

    id_sem = semget(clave, 1, 0666);
    if (id_sem == -1) {
        perror("Error al conectarse al semaforo");
        exit(1);
    }

    printf("\n-----Proceso [Pid=%d]-----\n", getpid());
    printf("Solicitando núcleo...\n");

    // WAIT (P) Solicitar un nucleo
    operacion.sem_num = 0;
    operacion.sem_op = -1;
    operacion.sem_flg = 0;
    semop(id_sem, &operacion, 1);

    disponibles = semctl(id_sem, 0, GETVAL);
    printf("Proceso obtuvo el nucleo | Disponibles: %d\n", disponibles);
}

```

```

// Simula que el nucleo esta trabajando
printf("\nProceso trabajando...\n");
sleep(50);

// SIGNAL (V) Liberar el núcleo
operacion.sem_op = 1;
semop(id_sem, &operacion, 1);

printf("Se libera el recurso\n");

return 0;
}

```

Realiza lo siguiente

- Ejecuta el semáforo contador (sc1)
- Revisa en la terminal se creo de manera correcta (ipcs -s)
- Abre 5 terminales.
- En cada una de ellas ejecuta el segundo programa (sc2)
- ¿Qué observas?
- ¿Cómo trabajan los semáforos contadores?



- Por buena practica se recomienda crear un programa limpiador que elimine el semáforo después de utilizarlo.
- como alternativa se puede hacer con la instrucción desde consola: **ipcrm -s idsemaforo**



```
#include<stdio.h>
#include<stdlib.h>
#include<sys/ipc.h>
#include<sys/sem.h>
int main(){
    key_t clave;
    int id_sem;
    clave = ftok("/tmp",'C');
    if(clave== -1){
        perror("Error al crear la llave");
        exit(1);
    }
    id_sem = semget(clave,1,0666);
    if(id_sem== -1){
        perror("Error al conectarse al semaforo");
        exit(1);
    }
    if(semctl(id_sem,0,IPC_RMID)== -1){
        perror("Error al eliminar el semaforo");
        exit(1);
    }
    printf("Semaforo eliminado correctamente.\n");
    return 0;
}
```

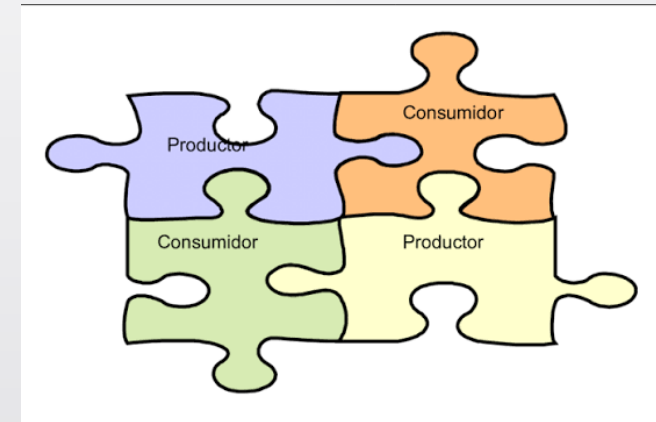
Problema del productor-consumidor

- Este problema puede implicar **dos procesos** o **múltiples productores y consumidores** que acceden al búfer compartido.
- El productor como el consumidor comparten un **búfer de tamaño fijo**, también llamado **búfer limitado**, que actúa como un **búfer común** en la memoria, compartido por **múltiples procesos** para su almacenamiento. El productor genera los datos en el búfer y el consumidor los consume.



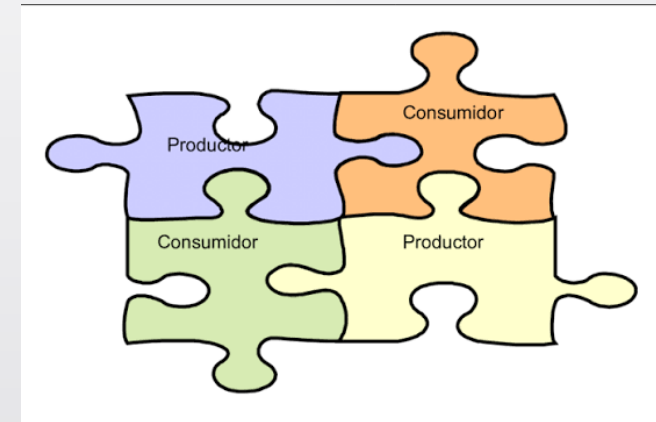
Problema del productor-consumidor

- El proceso productor no debe producir ningún dato cuando el búfer compartido esté **lleno** , porque cuando el **productor agrega** datos en ese momento puede causar **un desbordamiento del búfer** .
- El proceso consumidor no debe consumir ningún dato cuando el búfer compartido esté **vacío** , ya que cuando el **consumidor elimina** o lee datos, puede producirse **un desbordamiento negativo del búfer** .



Problema del productor-consumidor

- Durante **el acceso al búfer** , el acceso al búfer compartido debe ser mutuamente excluyente; es decir, solo un proceso a la vez debe poder acceder al búfer compartido y modificarlo. De lo contrario, pueden producirse **condiciones de carrera** cuando se actualizan simultáneamente **recursos compartidos** y una **variable compartida**, por lo que se requiere **una sincronización adecuada** .



Solución tres semáforos

- **Empty:** Indica cuántos espacios vacíos hay disponibles en el búfer. *Valor Inicial:* N (Tamaño máximo del búfer).
- **Full:** Indica cuántas ranuras del búfer están llenas de datos. *Valor Inicial:* 0 (Al principio no hay datos).
- **Mutex (Candado):** Garantiza la atomicidad de las operaciones sobre el almacenamiento. Se inicializa en 1. Asegura que **estrictamente un solo proceso** (ya sea el productor escribiendo o el consumidor leyendo)





Ejemplo

Escribe los códigos del productor-consumidor con tres semáforos

1

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#define TAM 3

union semun{
    int val; //se pueden omitir los otros valores (buf, array)
};

int main(){
    key_t clave;
    int id_sem, id_memoria, *buffer;
    union semun arg;
    struct sembuf operacion;

    clave=ftok("/tmp", 'P');

    if(clave==-1){
        perror("Error al crear la llave");
        exit(1);
    }

    id_memoria=shmget(clave, TAM*sizeof(int), IPC_CREAT|0666);
    if (id_memoria == -1) {
        perror("Error al crear la memoria compartida");
        exit(1);
    }

    buffer=(int *)shmat(id_memoria, NULL, 0);
    if (buffer == (int *)(-1)) {
        perror("Error al Adjuntar el buffer");
        exit(1);
    }
}

```

2

```

id_sem= semget(clave,3,IPC_CREAT|0666); //crear tres semaforos

if(id_sem == -1){
    perror("Error al crear el semaforo");
    exit(1);
}

//semaforo 1 empty (Representa cuántos espacios vacíos hay en el buffer)
arg.val=TAM; // Inicialmente el buffer está vacío
semctl(id_sem, 0, SETVAL, arg); // Hay TAM espacios disponibles

//semaforo 2 full (Representa cuántos items hay listos para consumir)
arg.val=0; //Inicialmente no hay paquetes para consumir
semctl(id_sem, 1, SETVAL, arg);

//semaforo 3, Binario (exclusión mutua entre el productor o consumidor)
arg.val=1; // Inicialmente el buffer está disponible (sin bloqueo)
semctl(id_sem, 2, SETVAL, arg); // Protege el acceso al buffer compartido

int numero_paquete=100;

```




```

for(int i=0; i<TAM; i++){
    //WAIT en empty, prueba si hay un espacio vacio
    operacion.sem_num=0;
    operacion.sem_op=-1;
    operacion.sem_flg=0;
    semop(id_sem, &operacion, 1);

    //WAIT en mutex, prueba entrar a la seccion critica
    operacion.sem_num=2;
    operacion.sem_op=-1;
    operacion.sem_flg=0;
    semop(id_sem, &operacion, 1);

    //seccion critica
    buffer[i]=numero_paquete;
    printf("PRODUCTOR\n Paquete %d guardado en el buffer[%d]\n", numero_paquete, i);

    //Mostrar el estado actual del buffer
    printf("Estado del buffer: ");
    for(int j=0; j<TAM; j++){
        printf("%d ", buffer[j]);
    }
    printf("\n");
    //FIN seccion critica

    //SIGNAL en mutex, salir de la seccion critica
    operacion.sem_num=2;
    operacion.sem_op=1;
    semop(id_sem, &operacion, 1);

    //SIGNAL en Full hay un item nuevo
    operacion.sem_num=1;
    operacion.sem_op= 1;
    operacion.sem_flg=0;
    semop(id_sem, &operacion, 1);
    numero_paquete++;
    sleep(1);
}

```



```

printf("\n Buffer lleno. Despues del consumidor regresa y presiona Enter\n");
getchar();

shmdt(buffer);
shmctl(id_memoria, IPC_RMID, NULL);
semctl(id_sem, 0, IPC_RMID);

printf("Memoria y semaforos eliminados\n");

return 0;
}

```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#define TAM 3
```

1

```
int main(){
    key_t clave;
    int id_sem, id_memoria, *buffer;
    struct sembuf operacion;

    clave=ftok("/tmp", 'P');

    if(clave==-1){
        perror("Error al crear la llave");
        exit(1);
    }

    id_memoria = shmget(clave, TAM*sizeof(int), 0666);
    if(id_memoria == -1){
        perror("Error al conectarse a la memoria compartida");
        exit(1);
    }

    buffer=(int *)shmat(id_memoria, NULL, 0);
    if (buffer == (int *)(-1)) {
        perror("Error al Adjuntar el buffer");
        exit(1);
    }

    //conectarse a los tres semaforos creados en el productor
    id_sem = semget(clave, 3, 0666);
    if (id_sem == -1) {
        perror("Error al conectarse a los semaforo");
        exit(1);
    }
}
```

```
printf("CONSUMIDOR\n\nEsperando paquetes disponibles...\n\n");
```

```
for(int i=0; i<TAM; i++){
```

```
    // WAIT (full) - Esperar paquete disponible
```

```
    operacion.sem_num=1;
```

```
    operacion.sem_op=-1;
```

```
    semop(id_sem, &operacion,1);
```

```
    // WAIT (mutex) - Proteger buffer
```

```
    operacion.sem_num=2;
```

```
    operacion.sem_op=-1;
```

```
    semop(id_sem, &operacion,1);
```

```
    //Region critica
```

```
    int paquete = buffer[i];
```

```
    buffer[i]=0; // Marcar casilla como vacía (solo para visualización)
```

```
    printf("Paquete %d extraído de buffer[%d]\n", paquete, i);
```

```
    //Mostrar el estado actual del buffer
```

```
    printf("Estado del buffer: ");
```

```
    for(int j=0; j<TAM; j++){
```

```
        printf("%d ", buffer[j]);
```

```
    }
```

```
    printf("\n");
```

```
    // SIGNAL (mutex) Liberar buffer
```

```
    operacion.sem_num=2;
```

```
    operacion.sem_op=1;
```

```
    semop(id_sem, &operacion,1);
```

```
    // SIGNAL (empty) - Avisar espacio libre
```

```
    operacion.sem_num=0;
```

```
    operacion.sem_op=1;
```

```
    semop(id_sem, &operacion,1);
```

```
    printf("Procesando paquete...\n");
```

```
    sleep(3);
```

```
}
```

2

consumidor.c

```
shmdt(buffer);  
printf("\nTodos los paquetes fueron consumidos\nConsumidor terminó\n");  
  
return 0;  
}
```

3

```
Terminal - alumnado@debianSO: ~/Documentos/programas  
Archivo Editar Ver Terminal Pestañas Ayuda  
alumnado@debianSO:~/Documentos/programas$ ./productor  
PRODUCTOR  
Paquete 100 guardado en el buffer[0]  
Estado del buffer: 100 0 0  
PRODUCTOR  
Paquete 101 guardado en el buffer[1]  
Estado del buffer: 100 101 0  
PRODUCTOR  
Paquete 102 guardado en el buffer[2]  
Estado del buffer: 100 101 102  
  
Buffer lleno. Despues del consumidor regresa y presiona Enter  
  
Memoria y semaforos eliminados  
alumnado@debianSO:~/Documentos/programas$
```

```
Terminal - alumnado@debianSO: ~/Documentos/programas  
Archivo Editar Ver Terminal Pestañas Ayuda  
alumnado@debianSO:~/Documentos/programas$ ./consumidor  
CONSUMIDOR  
  
Esperando paquetes disponibles...  
  
Paquete 100 extraído de buffer[0]  
Estado del buffer: 0 101 102  
Procesando paquete...  
Paquete 101 extraído de buffer[1]  
Estado del buffer: 0 0 102  
Procesando paquete...  
Paquete 102 extraído de buffer[2]  
Estado del buffer: 0 0 0  
Procesando paquete...  
  
Todos los paquetes fueron consumidos  
Consumidor terminó  
alumnado@debianSO:~/Documentos/programas$
```

productor.c

Actividad en clase

Modifique los programas de **Productor-Consumidor** desarrollados en clase para que realicen las siguientes tareas:

1. El **productor** deberá generar **cinco números aleatorios** en el rango de **1 a 200** y almacenarlos en el buffer compartido.
2. El **consumidor** deberá leer cada número del buffer y mostrar la siguiente información:
 - Indicar si el número es **par** o **impar**.
 - Determinar si el número es **primo**.
 - Calcular la **cantidad de divisores** que tiene.
 - Clasificar la **prioridad** del paquete de acuerdo con la siguiente tabla:

Rango	Prioridad
1 – 50	Alta
51 – 100	Media
101 – 200	Baja

Ejemplo

44

```
Terminal - alumnado@debianSO: ~/Documentos/programas
Archivo Editar Ver Terminal Pestañas Ayuda
alumnado@debianSO:~/Documentos/programas$ ./a6
PRODUCTOR

Paquete 37 almacenado en buffer[0]
Paquete 69 almacenado en buffer[1]
Paquete 112 almacenado en buffer[2]
Paquete 44 almacenado en buffer[3]
Paquete 130 almacenado en buffer[4]

Buffer lleno.
Ejecute el consumidor y posteriormente presione ENTER...
█
```

```
Terminal - alumnado@debianSO: ~/Documentos/pr
Archivo Editar Ver Terminal Pestañas Ayuda
alumnado@debianSO:~/Documentos/programas$ ./a6_2
CONSUMIDOR

Esperando paquetes disponibles...

-----
Paquete: 37
Par o impar: Impar
Primo: Si
Cantidad de divisores: 2
Prioridad: Alta
-----

-----
Paquete: 69
Par o impar: Impar
Primo: No
Cantidad de divisores: 4
Prioridad: Media
-----

-----
Paquete: 112
Par o impar: Par
Primo: No
Cantidad de divisores: 10
Prioridad: Baja
-----
```

```
-----
Paquete: 44
Par o impar: Par
Primo: No
Cantidad de divisores: 6
Prioridad: Alta
-----

-----
Paquete: 130
Par o impar: Par
Primo: No
Cantidad de divisores: 8
Prioridad: Baja
-----

Todos los paquetes fueron consumidos.
Consumidor terminó.
```